

# RAPPORT TECHNIQUE

## Système RAG — Puls-Events

*Proof of Concept — Chatbot de recommandation d'événements culturels en Île-de-France*

---

**Oumayma SELMEN**

Ingénieure Data — Mai 2026

## Table des matières

| Section | Titre                                        | Page |
|---------|----------------------------------------------|------|
| 1       | Introduction & Problématique                 | 3    |
| 2       | Environnement de travail & Livrables         | 4    |
| 3       | Architecture du Système                      | 5    |
| 4       | Choix Techniques & Justification des Modèles | 6    |
| 5       | Pipeline de Données & Scripts Documentés     | 8    |
| 6       | Vectorisation & Gestion de l'Index FAISS     | 10   |
| 7       | Système RAG & Chatbot                        | 11   |
| 8       | Tests Unitaires & Pipeline de Qualité        | 12   |
| 9       | Résultats du POC                             | 14   |
| 10      | Recommandations pour la Version Finale       | 15   |
| 11      | Conclusion                                   | 17   |
| Annexes | Structure projet · Commandes · Variables     | 18   |

## 1. Introduction

Puls-Events est une société innovante dans le domaine de la gestion d'événements culturels. Sa plateforme web permet aux utilisateurs de découvrir et de suivre des événements en temps réel. Dans le cadre de l'amélioration de l'expérience utilisateur, Puls-Events souhaite intégrer un chatbot intelligent capable de fournir des recommandations personnalisées basées sur des données réelles et actualisées.

Ce document présente le Proof of Concept (POC) d'un système RAG (Retrieval-Augmented Generation). Il décrit l'architecture technique, les choix d'implémentation, la documentation des scripts, les résultats obtenus et les recommandations pour la version finale.

### 1.1 Problématique

Les systèmes de recommandation traditionnels reposent sur des règles statiques. Un LLM classique ne connaît pas les événements actuels et risque d'inventer des réponses. La solution RAG connecte le LLM à une base de données réelle et actualisée.

#### Exemples de requêtes cibles

"Qu'est-ce qu'il y a à faire ce weekend à Paris pour les enfants ?" "Quels concerts de jazz y a-t-il en Île-de-France ce mois-ci ?"

### 1.2 Objectif du POC

- ▶ Récupérer des événements culturels en temps réel depuis OpenAgenda
- ▶ Indexer ces événements dans une base vectorielle FAISS
- ▶ Répondre en langage naturel via le modèle Mistral
- ▶ Obtenir un score de qualité supérieur à 85% sur un jeu de test annoté

### 1.3 KPIs du POC

**488**  
Événements collectés

**485**  
Après nettoyage

**89.6%**  
Score qualité

**5/5**  
Tests unitaires

## 2. Environnement de Travail & Livrables

### 2.1 Configuration de l'environnement

L'environnement de développement utilise un environnement virtuel Python (venv) pour garantir l'isolation des dépendances et la reproductibilité du projet.

```
# Setup de l'environnement
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
python test_imports.py         # Vérification imports
```

### 2.2 Gestion des dépendances (requirements.txt)

| Bibliothèque        | Version     | Rôle dans le projet                   |
|---------------------|-------------|---------------------------------------|
| langchain           | 0.3.14      | Orchestration du pipeline RAG         |
| langchain-community | 0.3.14      | Intégrations communautaires           |
| langchain-mistralai | 0.2.4       | Connecteur Mistral AI natif           |
| faiss-cpu           | 1.9.0.post1 | Base de données vectorielle locale    |
| mistralai           | 1.2.5       | Embeddings et génération de texte     |
| pandas              | 2.2.3       | Manipulation et nettoyage des données |
| python-dotenv       | 1.0.1       | Gestion des variables d'environnement |
| tiktoken            | 0.8.0       | Comptage des tokens                   |
| requests            | 2.32.3      | Appels API HTTP                       |
| pytest              | 9.0.3       | Tests unitaires automatisés           |

### 2.3 Livrables produits

| Livrable                          | Fichier(s)                           | Statut  |
|-----------------------------------|--------------------------------------|---------|
| Environnement + README            | venv/ · requirements.txt · README.md | ✓ Livré |
| Scripts pré-processing documentés | fetch_data.py · preprocess.py        | ✓ Livré |
| Scripts vectorisation + index     | vectorize.py · data/faiss_index      | ✓ Livré |
| Tests unitaires pipeline          | tests/test_data.py (5/5)             | ✓ Livré |
| Code système RAG                  | rag.py · chatbot.py                  | ✓ Livré |
| Jeu de données Q/R annoté         | qa_dataset.json · evaluate.py        | ✓ Livré |
| Rapport technique                 | rapport_technique.docx               | ✓ Livré |

### 3. Architecture du Système

#### 3.1 Vue d'ensemble

| Composant            | Rôle                                   | Technologie            |
|----------------------|----------------------------------------|------------------------|
| Collecte des données | Récupération des événements OpenAgenda | OpenDataSoft API       |
| Prétraitement        | Nettoyage et structuration des données | Pandas 2.2.3           |
| Découpage            | Fragmentation en chunks sémantiques    | LangChain TextSplitter |
| Vectorisation        | Transformation texte → vecteurs 1024D  | Mistral Embed          |
| Indexation           | Stockage et recherche vectorielle      | FAISS IndexFlatL2      |
| Orchestration        | Pipeline RAG complet                   | LangChain 0.3.14       |
| Génération           | Réponses en langage naturel            | Mistral Small Latest   |
| Interface            | Chatbot interactif CLI                 | Python 3.13            |

#### 3.2 Flux d'une requête utilisateur

##### Pipeline de traitement en 5 étapes

① Question utilisateur → ② Embedding (mistral-embed, 1024D) → ③ Recherche FAISS top-5 (distance L2) → ④ Récupération métadonnées → ⑤ Génération réponse (mistral-small)

## 4. Choix Techniques & Justification des Modèles

### 4.1 Source de données : OpenDataSoft

L'API officielle OpenAgenda v2 nécessite une autorisation spéciale non disponible pour ce POC. OpenDataSoft a été retenu comme alternative officielle :

- ▶ Portail public officiel hébergeant le dataset OpenAgenda (1M+ événements)
- ▶ Accès gratuit, sans restriction — adapté aux contraintes du POC
- ▶ API REST standard avec filtres avancés et pagination
- ▶ Extensible à toute la France pour la version production

### 4.2 Justification du modèle d'embedding : Mistral Embed

Conformément aux exigences du cahier des charges, le modèle d'embedding a été sélectionné selon les contraintes métier de Puls-Events.

| Critère                   | Mistral Embed ✓      | OpenAI Ada-002 | HuggingFace BGE  |
|---------------------------|----------------------|----------------|------------------|
| Performance français      | Excellente           | Bonne          | Moyenne          |
| Dimension vecteurs        | 1024D                | 1536D          | 768D             |
| Coût POC                  | Gratuit (Experiment) | Payant         | Gratuit local    |
| Intégration LangChain     | ✓ Native             | ✓ Native       | Manuelle         |
| Contrainte infrastructure | ✓ Cloud API          | Cloud API      | ✗ GPU recommandé |
| Adapté français           | ✓ Optimisé FR        | Généraliste    | Généraliste      |

#### Justification métier

Puls-Events opère en France. Un modèle optimisé pour le français garantit une meilleure compréhension sémantique des descriptions culturelles ("vernissage", "bal musette", "soirée guinguette"... ) et améliore directement la pertinence des recommandations.

### 4.3 Justification du modèle de génération : Mistral Small

- ▶ Optimisé français — priorité métier pour Puls-Events
- ▶ Même provider que mistral-embed — cohérence technique et facturation
- ▶ Équilibre qualité / coût / latence optimal pour le POC
- ▶ Conforme au cahier des charges (intégration LangChain + Mistral)

### 4.4 Comparaison des bases vectorielles

| Critère                   | FAISS ✓             | Chroma              | Pinecone       |
|---------------------------|---------------------|---------------------|----------------|
| Type                      | Local / Open Source | Local / Open Source | Cloud managé   |
| Coût                      | ✓ Gratuit           | ✓ Gratuit           | ✗ Payant       |
| Vitesse (< 1M vecteurs)   | ✓ Très rapide       | Rapide              | Rapide         |
| Scalabilité (> 1M)        | Limite (IVF requis) | Bonne               | ✓ Excellente   |
| Intégration LangChain     | ✓ Native            | ✓ Native            | ✓ Native       |
| Adapté POC (495 vecteurs) | ✓ Optimal           | Possible            | Surdimensionné |

## 5. Pipeline de Données & Scripts Documentés

### 5.1 fetch\_data.py — Récupération des données

Script de récupération avec pagination automatique et gestion des erreurs API.

```
def fetch_events(region: str = 'Île-de-France', max_records: int = 500) ->
pd.DataFrame:
    """
    Récupère les événements culturels depuis l'API OpenDataSoft.

    Args:
        region (str): Région géographique cible. Défaut: 'Île-de-France'
        max_records (int): Nombre maximum d'événements. Défaut: 500

    Returns:
        pd.DataFrame: DataFrame des événements filtrés (région + 12 mois)

    Raises:
        requests.exceptions.RequestException: Si l'API est inaccessible
    """
```

### 5.2 preprocess.py — Nettoyage des données

Script de nettoyage avec filtre temporel strict et création de la colonne d'embedding.

```
def preprocess_events(input_path: str, output_path: str) -> pd.DataFrame:
    """
    Nettoie et prépare les événements pour la vectorisation.

    Opérations:
        1. Conversion et validation des dates (pd.to_datetime)
        2. Filtrage strict: événements des 12 derniers mois uniquement
        3. Suppression doublons (title_fr + firstdate_begin)
        4. Suppression valeurs manquantes (title_fr, description_fr)
        5. Création colonne 'text_for_embedding' structurée

    Returns:
        pd.DataFrame: 485 événements propres prêts à vectoriser
    """
```

### 5.3 vectorize.py — Vectorisation & Indexation

```
def build_faiss_index(input_path: str, index_path: str, metadata_path:
str):
    """
```

```
Construit la base vectorielle FAISS depuis les données nettoyées.
```

```
Pipeline interne:
```

1. Chargement events\_clean.json (485 événements)
2. Découpage: RecursiveCharacterTextSplitter(size=500, overlap=50)
3. Vectorisation: mistral-embed par batches de 10 → 49 batches
4. Indexation: FAISS IndexFlatL2 (dim=1024, 495 vecteurs)
5. Sauvegarde: faiss\_index + metadata.json

```
Returns:
```

```
None — fichiers sauvegardés dans data/
```

```
"""
```

## 6. Vectorisation & Gestion de l'Index FAISS

### 6.1 Résultats de la vectorisation

| Métrique               | Valeur                                |
|------------------------|---------------------------------------|
| Événements en entrée   | 485                                   |
| Chunks créés           | 495 (RecursiveCharacterTextSplitter)  |
| Taille chunk           | 500 caractères / 50 overlap           |
| Modèle d'embedding     | mistral-embed                         |
| Dimension des vecteurs | 1024                                  |
| Nombre de batches      | 49 (10 textes/batch)                  |
| Type d'index FAISS     | IndexFlatL2 (distance euclidienne)    |
| Fichiers produits      | data/faiss_index + data/metadata.json |

### 6.2 Reconstruction de l'index sur demande

L'index FAISS peut être reconstruit intégralement à tout moment :

```
# Reconstruction complète de la base vectorielle
python src/fetch_data.py      # 1. Collecte fraîche OpenDataSoft
python src/preprocess.py      # 2. Nettoyage + filtrage
python src/vectorize.py        # 3. Vectorisation + indexation FAISS
python -m pytest tests/test_data.py -v # 4. Validation automatique
```

## 7. Système RAG & Chatbot

### 7.1 rag.py — Orchestration LangChain

LangChain orchestre les interactions entre FAISS et Mistral. Ce choix est justifié par :

- ▶ Intégration native avec FAISS et Mistral AI — pas de développement custom
- ▶ RecursiveCharacterTextSplitter optimisé pour le découpage sémantique
- ▶ Abstraction des appels API — facilite l'évolution vers d'autres modèles
- ▶ Conformité aux exigences du cahier des charges

### 7.2 chatbot.py — Interface CLI

Interface interactive en boucle continue. Conformément aux exigences du POC, l'historique de conversation n'est pas implémenté à ce stade — il sera ajouté en version production via ConversationBufferMemory de LangChain.

## 8. Tests Unitaires & Pipeline de Qualité

### 8.1 Pipeline de validation automatique

Les tests unitaires sont intégrés dans le pipeline de reconstruction. Ils s'exécutent automatiquement après chaque mise à jour des données pour garantir la qualité en continu.

```
# Pipeline complet avec validation intégrée
python src/fetch_data.py
python src/preprocess.py
python src/vectorize.py
python -m pytest tests/test_data.py -v      # Validation pipeline
python tests/evaluate.py                    # Score qualité RAG
```

### 8.2 Tests unitaires — Résultats

| Test                       | Vérification                                              | Résultat |
|----------------------------|-----------------------------------------------------------|----------|
| test_data_not_empty        | Le dataset contient des événements (len > 0)              | ✓ PASSED |
| test_region_is_idf         | Tous les événements ont location_region = 'Île-de-France' | ✓ PASSED |
| test_dates_within_one_year | Toutes les dates firstdate_begin sont < 1 an              | ✓ PASSED |
| test_no_missing_titles     | Aucune valeur nulle dans la colonne title_fr              | ✓ PASSED |
| test_metadata_matches_data | len(metadata) >= len(df) — cohérence FAISS/JSON           | ✓ PASSED |

### 8.3 Méthodes de garantie de la qualité des données

- ▶ Filtrage à la source : paramètres API stricts (région + période)
- ▶ Validation des types : conversion et vérification dates avec Pandas
- ▶ Déduplication : suppression sur (title\_fr, firstdate\_begin)
- ▶ Contrôle valeurs manquantes : suppression événements sans titre/description
- ▶ Tests automatiques après chaque reconstruction du pipeline
- ▶ Évaluation sémantique : similarité cosinus sur jeu de 5 Q/R annotées

### 8.4 Évaluation de la qualité des réponses — qa\_dataset.json

Un jeu de données test annoté a été créé pour mesurer la qualité des réponses. La méthode utilise la similarité cosinus entre les embeddings Mistral des réponses générées et attendues.

#### Formule d'évaluation

Score =  $\text{cosine\_similarity}(\text{embedding}(\text{réponse\_chatbot}), \text{embedding}(\text{réponse\_attendue})) \times 100$

Seuil cible POC : > 85% — Score obtenu : 89.6% ✓

## 9. Résultats du POC

### 9.1 Données collectées

| Métrique                    | Valeur                   |
|-----------------------------|--------------------------|
| Événements récupérés (brut) | 488                      |
| Événements après nettoyage  | 485                      |
| Région géographique         | Île-de-France            |
| Période couverte            | 12 derniers mois         |
| Chunks créés                | 495                      |
| Dimension des vecteurs      | 1024                     |
| Tests unitaires passés      | 5 / 5                    |
| Score qualité moyen         | 89.6% (objectif : 85%) ✓ |

### 9.2 Évaluation détaillée — Similarité Cosinus

| Question test                     | Score obtenu | Statut             |
|-----------------------------------|--------------|--------------------|
| Concerts de jazz à Paris          | 90.8%        | ✓ Excellent        |
| Événements pour enfants à Paris   | 89.9%        | ✓ Excellent        |
| Expositions en Île-de-France      | 90.8%        | ✓ Excellent        |
| Spectacles de théâtre à Paris     | 88.0%        | ✓ Bon              |
| Activités nature en Île-de-France | 88.5%        | ✓ Bon              |
| Score moyen                       | 89.6%        | ✓ Objectif dépassé |

## 10. Recommandations pour la Version Finale

### 10.1 Améliorations techniques

- ▶ Remplacer IndexFlatL2 par IndexIVFFlat pour scalabilité > 100k événements
- ▶ Mettre en place une mise à jour automatique quotidienne de la base vectorielle
- ▶ Ajouter l'historique de conversation (LangChain ConversationBufferMemory)
- ▶ Implémenter une API REST (FastAPI) pour exposer le chatbot
- ▶ Déployer sur cloud AWS Lambda + S3 ou GCP Cloud Run

### 10.2 Détection des dérives de performance en production

- ▶ Score de similarité moyen sur jeu de test → alerte si < 80%
- ▶ Taux de réponses hors sujet → alerte si > 10%
- ▶ Latence de réponse P95 → alerte si > 3 secondes
- ▶ Fraîcheur des données → alerte si âge moyen > 30 jours
- ▶ Réévaluation mensuelle sur un jeu de test élargi (50+ questions)

### 10.3 Indicateurs de performance clés (KPIs production)

| Indicateur             | Description                                   | Seuil cible  |
|------------------------|-----------------------------------------------|--------------|
| Score similarité moyen | Qualité sémantique des réponses (cosinus)     | > 85%        |
| Latence P95            | Temps de réponse au 95e percentile            | < 3 secondes |
| Taux de couverture     | Requêtes avec au moins 3 résultats pertinents | > 90%        |
| Fraîcheur des données  | Âge moyen des événements indexés              | < 30 jours   |
| Disponibilité API      | Uptime du service chatbot                     | > 99.5%      |
| Taux de réponses vides | Requêtes sans résultat FAISS                  | < 5%         |

### 10.4 Estimation des ressources

| Ressource               | POC (actuel) | Version Finale  |
|-------------------------|--------------|-----------------|
| Événements indexés      | 485          | 50 000+         |
| Index FAISS             | IndexFlatL2  | IndexIVFFlat    |
| Infrastructure          | Local        | Cloud (AWS/GCP) |
| Coût mensuel estimé     | < 10€        | 100–500€        |
| Temps de reconstruction | ~5 minutes   | ~30 minutes     |

## 11. Conclusion

Ce POC démontre la faisabilité d'un système RAG pour la recommandation d'événements culturels. Les résultats obtenus valident l'approche technique et dépassent les objectifs fixés :

**89.6%**

Score qualité moyen

**5/5**

Tests unitaires

**495**

Chunks indexés

**< 10€**

Coût mensuel POC

- ▶ Pipeline complet opérationnel de la collecte à la génération
- ▶ Architecture modulaire avec scripts entièrement documentés (docstrings)
- ▶ Modèles sélectionnés et justifiés selon les contraintes métier
- ▶ Tests unitaires intégrés dans le pipeline de reconstruction
- ▶ Jeu de données Q/R annoté pour évaluation continue de la qualité

Le système est prêt pour une présentation aux équipes produit et marketing. Les recommandations formulées permettront de passer à une version production robuste, scalable et monitorée.

## Annexes

### A. Structure du projet

```
projet 11/
├─ src/
│   ├── fetch_data.py      # Collecte OpenDataSoft (docstrings)
│   ├── preprocess.py      # Nettoyage Pandas (docstrings)
│   ├── vectorize.py       # Vectorisation + FAISS (docstrings)
│   ├── rag.py             # Pipeline RAG LangChain
│   └─ chatbot.py          # Interface CLI
├─ tests/
│   ├── test_data.py       # 5 tests unitaires pytest
│   ├── qa_dataset.json    # Jeu Q/R annoté
│   ├── evaluate.py        # Évaluation similarité cosinus
│   └─ resultats_evaluation.json
├─ data/
│   ├── raw/
│   │   ├── events_idf.json
│   │   └─ events_clean.json
│   ├── faiss_index
│   └─ metadata.json
├─ .env                    # Variables d'environnement
├─ requirements.txt
├─ README.md
└─ test_imports.py
```

### B. Variables d'environnement (.env)

```
OPENAGENDA_API_KEY=<clé_publicque_openagenda>
MISTRAL_API_KEY=<clé_api_mistral>
```

### C. Commandes de lancement

```
# 1. Activer l'environnement
venv\Scripts\activate

# 2. Reconstruire la base vectorielle (pipeline complet)
python src/fetch_data.py
python src/preprocess.py
python src/vectorize.py

# 3. Valider les données (pipeline de tests)
python -m pytest tests/test_data.py -v

# 4. Lancer le chatbot
```

```
python src/chatbot.py

# 5. Évaluer la qualité des réponses
python tests/evaluate.py
```